

SCR2043 OPERATING SYSTEMS

Name : Lam Yoke Yu
Student ID : A23CS0233
Section : 03

Marks

This lab assessment is designed to test your understanding and skills on some basic concepts and tools related to process monitoring and management in operating system. Please follow the instructions carefully and submit your answers in this word document and rename the file as **os-lab-assessment02-studentname-matricno.docx**.

Essential Steps Before Starting Lab Assessment 2:

1. Download necessary source codes:

Use the `wget` command to retrieve the following source code files to your Linux (or WSL or MacOS) environment:

```
wget -O mainprocess.c https://rebrand.ly/mainprocess_c
wget -O subprocess1.c https://rebrand.ly/subprocess1_c
wget -O subprocess2.c https://rebrand.ly/subprocess2_c
```

2. Compile the source files:

Use the `gcc` compiler to create executable files from the source code.

```
gcc mainprocess.c -o mainprocess
gcc subprocess1.c -o subprocess1
gcc subprocess2.c -o subprocess2
```

3. Execute the dummy processes:

Run all the dummy processes

```
./mainprocess &
```

Press **enter** two times.

4. The dummy processes are running for 2 hours. If you took longer than 2 hours on questions 1-9, please restart the main process with `./mainprocess &`.

Lab Assessment 2 : Linux Process Monitoring and Management

Instructions:

1. Carefully execute each command as instructed in the questions.
2. Write down the exact command used for each task.
3. Capture a screenshot of the command's output.

Question 1

Use the `ps` command with the appropriate option to display a complete list of all running processes within the Linux operating system.

Command			
ps -e			
Output			
PID	TTY	TIME	CMD
1	?	00:00:04	systemd
2	?	00:00:00	kthreadd
3	?	00:00:00	pool_workqueue_release
4	?	00:00:00	kworker/R-rcu_g
5	?	00:00:00	kworker/R-rcu_p
6	?	00:00:00	kworker/R-slub_
7	?	00:00:00	kworker/R-netns
10	?	00:00:00	kworker/0:0H-kblockd
11	?	00:00:00	kworker/u4:0-ext4-rsv-conversion
12	?	00:00:00	kworker/R-mm_pe
13	?	00:00:00	rcu_tasks_kthread
14	?	00:00:00	rcu_tasks_rude_kthread
15	?	00:00:00	rcu_tasks_trace_kthread
16	?	00:00:00	ksoftirqd/0
17	?	00:00:00	rcu_preempt
18	?	00:00:00	migration/0
19	?	00:00:00	idle_inject/0
20	?	00:00:00	cpuhp/0
21	?	00:00:00	cpuhp/1
22	?	00:00:00	idle_inject/1
23	?	00:00:00	migration/1
24	?	00:00:00	ksoftirqd/1
26	?	00:00:00	kworker/1:0H-events_highpri
27	?	00:00:00	kworker/u5:0-events_power_efficient
29	?	00:00:00	kdevtmpfs
30	?	00:00:00	kworker/R-inet_
31	?	00:00:00	kworker/u5:1-events_power_efficient
32	?	00:00:00	kauditd
33	?	00:00:00	khungtaskd
34	?	00:00:00	oom_reaper
35	?	00:00:00	kworker/u5:2-events_unbound
36	?	00:00:00	kworker/R-write
37	?	00:00:00	kcompactd0
38	?	00:00:00	ksmd
39	?	00:00:00	kworker/1:1-cgwb_release
40	?	00:00:00	khugepaged
41	?	00:00:00	kworker/R-kinte
42	?	00:00:00	kworker/R-kbloc
43	?	00:00:00	kworker/R-blkcg
44	?	00:00:00	irq/9-acpi

45 ?	00:00:00	kworker/R-tpm_d
46 ?	00:00:00	kworker/R-ata_s
47 ?	00:00:00	kworker/R-md
48 ?	00:00:00	kworker/R-md_bi
49 ?	00:00:00	kworker/R-edac-
50 ?	00:00:00	kworker/R-devfr
51 ?	00:00:00	watchdogd
52 ?	00:00:00	kworker/u6:1-events_unbound
53 ?	00:00:00	kworker/1:1H-kblockd
54 ?	00:00:00	kswapd0
55 ?	00:00:00	ecryptfs-kthread
56 ?	00:00:00	kworker/R-kthro
57 ?	00:00:00	kworker/R-acpi_
58 ?	00:00:00	scsi_eh_0
59 ?	00:00:00	kworker/R-scsi_
60 ?	00:00:00	scsi_eh_1
61 ?	00:00:00	kworker/R-scsi_
62 ?	00:00:00	kworker/0:1H-kblockd
63 ?	00:00:00	kworker/R-mld
64 ?	00:00:00	kworker/R-ipv6_
72 ?	00:00:00	kworker/R-kstrp
74 ?	00:00:00	kworker/u7:0
75 ?	00:00:00	kworker/u8:0
76 ?	00:00:00	kworker/u9:0
82 ?	00:00:00	kworker/R-crypt
83 ?	00:00:00	kworker/0:2-events
92 ?	00:00:00	kworker/R-charg
133 ?	00:00:03	kworker/0:3-events
134 ?	00:00:00	kworker/u4:1
136 ?	00:00:00	kworker/R-mpt_p
137 ?	00:00:00	kworker/R-mpt/0
156 ?	00:00:00	scsi_eh_2
157 ?	00:00:00	kworker/R-scsi_
191 ?	00:00:00	kworker/R-raid5
228 ?	00:00:00	jbd2/sda1-8
229 ?	00:00:00	kworker/R-ext4-
296 ?	00:00:00	psimon
301 ?	00:00:00	systemd-journal
322 ?	00:00:00	kworker/R-kmpat
323 ?	00:00:00	kworker/R-kmpat
324 ?	00:00:00	kworker/u6:3-events_power_efficient
359 ?	00:00:00	multipathd
370 ?	00:00:00	systemd-udevd
371 ?	00:00:01	kworker/1:4-events
379 ?	00:00:00	psimon
455 ?	00:00:00	irq/18-vmwgfx
456 ?	00:00:00	kworker/R-ttm
478 ?	00:00:00	jbd2/sda16-8
479 ?	00:00:00	kworker/R-ext4-
511 ?	00:00:00	systemd-network
528 ?	00:00:00	systemd-resolve
533 ?	00:00:00	systemd-timesyn
636 ?	00:00:00	dbus-daemon
641 ?	00:00:00	polkitd
651 ?	00:00:00	systemd-logind
657 ?	00:00:00	udisksd
702 ?	00:00:00	unattended-upgr
707 ?	00:00:00	rsyslogd
735 ?	00:00:00	ModemManager

775	?	00:00:00	cron
804	tty1	00:00:00	agetty
865	?	00:00:00	sshd
906	?	00:00:00	sshd
908	?	00:00:01	sshd
909	pts/0	00:00:00	bash
951	pts/0	00:00:00	mainprocess
952	pts/0	00:00:00	mainprocess
953	pts/0	00:00:00	mainprocess
954	pts/0	00:00:00	subprocess2
955	pts/0	00:00:00	subprocess2
956	pts/0	00:00:00	subprocess2
957	pts/0	00:00:00	subprocess1
958	pts/0	00:00:00	subprocess1
964	?	00:00:00	kworker/u6:0-flush-8:0
965	?	00:00:00	kworker/1:0
966	pts/0	00:00:00	ps

Question 2

Employ the `ps` command with necessary options to unveil comprehensive details about each running process.

Command							
ps -ef							
Output							
UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	1	0	0	02:43	?	00:00:04	/sbin/init
root	2	0	0	02:43	?	00:00:00	[kthreadd]
root	3	2	0	02:43	?	00:00:00	[pool_workqu
root	4	2	0	02:43	?	00:00:00	[kworker/R-r
root	5	2	0	02:43	?	00:00:00	[kworker/R-r
root	6	2	0	02:43	?	00:00:00	[kworker/R-s
root	7	2	0	02:43	?	00:00:00	[kworker/R-n
root	10	2	0	02:43	?	00:00:00	[kworker/0:0
root	11	2	0	02:43	?	00:00:00	[kworker/u4:
root	12	2	0	02:43	?	00:00:00	[kworker/R-m
root	13	2	0	02:43	?	00:00:00	[rcu_tasks_k
root	14	2	0	02:43	?	00:00:00	[rcu_tasks_r
root	15	2	0	02:43	?	00:00:00	[rcu_tasks_t
root	16	2	0	02:43	?	00:00:00	[ksoftirqd/0
root	17	2	0	02:43	?	00:00:00	[rcu_preempt
root	18	2	0	02:43	?	00:00:00	[migration/0
root	19	2	0	02:43	?	00:00:00	[idle_inject
root	20	2	0	02:43	?	00:00:00	[cpuhp/0]
root	21	2	0	02:43	?	00:00:00	[cpuhp/1]
root	22	2	0	02:43	?	00:00:00	[idle_inject
root	23	2	0	02:43	?	00:00:00	[migration/1
root	24	2	0	02:43	?	00:00:00	[ksoftirqd/1
root	26	2	0	02:43	?	00:00:00	[kworker/1:0
root	27	2	0	02:43	?	00:00:00	[kworker/u5:
root	29	2	0	02:43	?	00:00:00	[kdevtmpfs]
root	30	2	0	02:43	?	00:00:00	[kworker/R-i
root	31	2	0	02:43	?	00:00:00	[kworker/u5:
root	32	2	0	02:43	?	00:00:00	[kauditd]
root	33	2	0	02:43	?	00:00:00	[khungtaskd]
root	34	2	0	02:43	?	00:00:00	[oom_reaper]

root	35	2	0	02:43	?	00:00:00	[kworker/u5:
root	36	2	0	02:43	?	00:00:00	[kworker/R-w
root	37	2	0	02:43	?	00:00:00	[kcompactd0]
root	38	2	0	02:43	?	00:00:00	[ksmd]
root	39	2	0	02:43	?	00:00:00	[kworker/1:1
root	40	2	0	02:43	?	00:00:00	[khugepaged]
root	41	2	0	02:43	?	00:00:00	[kworker/R-k
root	42	2	0	02:43	?	00:00:00	[kworker/R-k
root	43	2	0	02:43	?	00:00:00	[kworker/R-b
root	44	2	0	02:43	?	00:00:00	[irq/9-acpi]
root	45	2	0	02:43	?	00:00:00	[kworker/R-t
root	46	2	0	02:43	?	00:00:00	[kworker/R-a
root	47	2	0	02:43	?	00:00:00	[kworker/R-m
root	48	2	0	02:43	?	00:00:00	[kworker/R-m
root	49	2	0	02:43	?	00:00:00	[kworker/R-e
root	50	2	0	02:43	?	00:00:00	[kworker/R-d
root	51	2	0	02:43	?	00:00:00	[watchdogd]
root	52	2	0	02:43	?	00:00:00	[kworker/u6:
root	53	2	0	02:43	?	00:00:00	[kworker/1:1
root	54	2	0	02:43	?	00:00:00	[kswapd0]
root	55	2	0	02:43	?	00:00:00	[ecryptfs-kt
root	56	2	0	02:43	?	00:00:00	[kworker/R-k
root	57	2	0	02:43	?	00:00:00	[kworker/R-a
root	58	2	0	02:43	?	00:00:00	[scsi_eh_0]
root	59	2	0	02:43	?	00:00:00	[kworker/R-s
root	60	2	0	02:43	?	00:00:00	[scsi_eh_1]
root	61	2	0	02:43	?	00:00:00	[kworker/R-s
root	62	2	0	02:43	?	00:00:00	[kworker/0:1
root	63	2	0	02:43	?	00:00:00	[kworker/R-m
root	64	2	0	02:43	?	00:00:00	[kworker/R-i
root	72	2	0	02:43	?	00:00:00	[kworker/R-k
root	74	2	0	02:43	?	00:00:00	[kworker/u7:
root	75	2	0	02:43	?	00:00:00	[kworker/u8:
root	76	2	0	02:43	?	00:00:00	[kworker/u9:
root	82	2	0	02:43	?	00:00:00	[kworker/R-c
root	83	2	0	02:43	?	00:00:00	[kworker/0:2
root	92	2	0	02:43	?	00:00:00	[kworker/R-c
root	133	2	0	02:43	?	00:00:04	[kworker/0:3
root	134	2	0	02:43	?	00:00:00	[kworker/u4:
root	136	2	0	02:43	?	00:00:00	[kworker/R-m
root	137	2	0	02:43	?	00:00:00	[kworker/R-m
root	156	2	0	02:43	?	00:00:00	[scsi_eh_2]
root	157	2	0	02:43	?	00:00:00	[kworker/R-s
root	191	2	0	02:43	?	00:00:00	[kworker/R-r
root	228	2	0	02:43	?	00:00:00	[jbd2/sda1-8
root	229	2	0	02:43	?	00:00:00	[kworker/R-e
root	296	2	0	02:43	?	00:00:00	[psimon]
root	301	1	0	02:43	?	00:00:00	/usr/lib/sys
root	322	2	0	02:43	?	00:00:00	[kworker/R-k
root	323	2	0	02:43	?	00:00:00	[kworker/R-k
root	324	2	0	02:43	?	00:00:00	[kworker/u6:
root	359	1	0	02:43	?	00:00:00	/sbin/multip
root	370	1	0	02:43	?	00:00:00	/usr/lib/sys
root	371	2	0	02:43	?	00:00:01	[kworker/1:4
root	379	2	0	02:43	?	00:00:00	[psimon]
root	455	2	0	02:43	?	00:00:00	[irq/18-vmwg
root	456	2	0	02:43	?	00:00:00	[kworker/R-t
root	478	2	0	02:43	?	00:00:00	[jbd2/sda16-
root	479	2	0	02:43	?	00:00:00	[kworker/R-e

systemd+	511	1	0	02:43	?	00:00:00	/usr/lib/sys
systemd+	528	1	0	02:43	?	00:00:00	/usr/lib/sys
systemd+	533	1	0	02:43	?	00:00:00	/usr/lib/sys
message+	636	1	0	02:43	?	00:00:00	@dbus-daemon
polkitd	641	1	0	02:43	?	00:00:00	/usr/lib/pol
root	651	1	0	02:43	?	00:00:00	/usr/lib/sys
root	657	1	0	02:43	?	00:00:00	/usr/libexec
root	702	1	0	02:43	?	00:00:00	/usr/bin/pyt
syslog	707	1	0	02:43	?	00:00:00	/usr/sbin/rs
root	735	1	0	02:43	?	00:00:00	/usr/sbin/Mo
root	775	1	0	02:43	?	00:00:00	/usr/sbin/cr
root	804	1	0	02:43	tty1	00:00:00	/sbin/agetty
root	865	1	0	02:44	?	00:00:00	sshd: /usr/s
root	906	865	0	02:45	?	00:00:00	sshd: lam [p
lam	908	906	0	02:46	?	00:00:01	sshd: lam@pt
lam	909	908	0	02:46	pts/0	00:00:00	-bash
lam	951	909	0	02:47	pts/0	00:00:00	./mainproces
lam	952	951	0	02:47	pts/0	00:00:00	./mainproces
lam	953	951	0	02:47	pts/0	00:00:00	./mainproces
lam	954	953	0	02:47	pts/0	00:00:00	./subprocess
lam	955	953	0	02:47	pts/0	00:00:00	./subprocess
lam	956	953	0	02:47	pts/0	00:00:00	./subprocess
lam	957	952	0	02:47	pts/0	00:00:00	./subprocess
lam	958	952	0	02:47	pts/0	00:00:00	./subprocess
root	964	2	0	02:49	?	00:00:00	[kworker/u6:
root	965	2	0	02:49	?	00:00:00	[kworker/1:0
root	969	2	0	02:50	?	00:00:00	[kworker/u5:
lam	970	909	99	02:52	pts/0	00:00:00	ps -ef

Question 3

Use the `ps` command with some tools to only list processes named "subprocess" and show some info about them.

Command							
<code>ps -ef grep '[s]ubprocess'</code>							
Output							
lam@secr2043:~\$ ps -ef grep '[s]ubprocess'							
lam	954	953	0	02:47	pts/0	00:00:00	./subprocess2
lam	955	953	0	02:47	pts/0	00:00:00	./subprocess2
lam	956	953	0	02:47	pts/0	00:00:00	./subprocess2
lam	957	952	0	02:47	pts/0	00:00:00	./subprocess1
lam	958	952	0	02:47	pts/0	00:00:00	./subprocess1

Question 4

Execute the `ps` command, specifying options that reveal only the following columns:

- Process ID (`pid`)
- Owner of the process (`user`)
- CPU percentage (`pcpu`)
- Memory percentage (`pmem`)

- Command (cmd)

Command				
ps -eo pid,user,pcpu,pmem,cmd grep '[s]ubprocess'				
Output				
lam@secr2043:~\$ ps -eo pid,user,pcpu,pmem,cmd grep '[s]ubprocess'				
954	lam	0.0	0.1	./subprocess2
955	lam	0.0	0.1	./subprocess2
956	lam	0.0	0.1	./subprocess2
957	lam	0.0	0.1	./subprocess1
958	lam	0.0	0.1	./subprocess1

Question 5

Building on the ps command used in Question 4, can you add an option to sort the listed processes by their memory usage (pmem)?

Command				
ps -eo pid,user,pcpu,pmem,cmd --sort=pmem grep '[s]ubprocess'				
Output				
lam@secr2043:~\$ ps -eo pid,user,pcpu,pmem,cmd --sort=pmem grep '[s]ubprocess'				
954	lam	0.0	0.1	./subprocess2
955	lam	0.0	0.1	./subprocess2
956	lam	0.0	0.1	./subprocess2
957	lam	0.0	0.1	./subprocess1
958	lam	0.0	0.1	./subprocess1

Question 6

Construct a command using ps, suitable options, and any additional tools to visualize the hierarchical structure (tree-like) of the following processes:

- "mainprocess"
- "subprocess1"
- "subprocess2"

Command																																					
ps -eo pid,ppid,user,cmd --forest grep -E 'mainprocess subprocess1 subprocess2'																																					
Output																																					
<pre>lam@secr2043:~\$ ps -eo pid,ppid,user,cmd --forest grep -E 'mainprocess subprocess1 subprocess2'</pre> <table><tr><td>951</td><td>909</td><td>lam</td><td>_ ./mainprocess</td></tr><tr><td>952</td><td>951</td><td>lam</td><td> _ ./mainprocess</td></tr><tr><td>957</td><td>952</td><td>lam</td><td> _ ./subprocess1</td></tr><tr><td>958</td><td>952</td><td>lam</td><td> _ ./subprocess1</td></tr><tr><td>953</td><td>951</td><td>lam</td><td> _ ./mainprocess</td></tr><tr><td>954</td><td>953</td><td>lam</td><td> _ ./subprocess2</td></tr><tr><td>955</td><td>953</td><td>lam</td><td> _ ./subprocess2</td></tr><tr><td>956</td><td>953</td><td>lam</td><td> _ ./subprocess2</td></tr><tr><td>1014</td><td>909</td><td>lam</td><td>_ grep --color=auto -E mainprocess subprocess1 subprocess2</td></tr></table>		951	909	lam	_ ./mainprocess	952	951	lam	_ ./mainprocess	957	952	lam	_ ./subprocess1	958	952	lam	_ ./subprocess1	953	951	lam	_ ./mainprocess	954	953	lam	_ ./subprocess2	955	953	lam	_ ./subprocess2	956	953	lam	_ ./subprocess2	1014	909	lam	_ grep --color=auto -E mainprocess subprocess1 subprocess2
951	909	lam	_ ./mainprocess																																		
952	951	lam	_ ./mainprocess																																		
957	952	lam	_ ./subprocess1																																		
958	952	lam	_ ./subprocess1																																		
953	951	lam	_ ./mainprocess																																		
954	953	lam	_ ./subprocess2																																		
955	953	lam	_ ./subprocess2																																		
956	953	lam	_ ./subprocess2																																		
1014	909	lam	_ grep --color=auto -E mainprocess subprocess1 subprocess2																																		

Question 7

Use `pstree` command with option that show the number of threads to each process.

Command
<code>pstree -T</code>
Output
<pre>lam@secr2043:~\$ pstree -T systemd--+-ModemManager -agetty -cron -dbus-daemon -multipathd -polkitd -rsyslogd -sshd---sshd---sshd---bash--+-mainprocess--+-mainprocess---2*[subprocess1] \-mainprocess---3*[subprocess2] \-pstree -systemd-journal -systemd-logind -systemd-network -systemd-resolve -systemd-timesyn -systemd-udev -udisksd \-unattended-upgr lam@secr2043:~\$ pstree -c -s 951 systemd---sshd---sshd---sshd---bash---mainprocess--+-mainprocess--+-subprocess1 \-subprocess1 \-mainprocess--+-subprocess2 \-subprocess2 \-subprocess2</pre>

Question 8

Use `renice` command to change priority level of one of process “subprocess1”.

Command
<code>sudo renice -n -10 -p 957</code>
Output
<pre>lam@secr2043:~\$ sudo renice -n -10 -p 957 [sudo] password for lam: 957 (process ID) old priority 0, new priority -10</pre>

Question 9

Terminate all running processes with the name “mainprocess”.

Command
killall -15 mainprocess
Output
lam@secr2043:~\$ killall -15 mainprocess Main process (ID: 952) received signal: 15. Terminating... Main process (ID: 951) received signal: 15. Terminating...

Question 10

Write a short C or Python code (choose only one language) demonstrating multiprocessing with `fork()` and `wait()`. Compile and/or run the code. Show the output.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

int main(){
    pid_t pid;

    /*fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /*child process */
        printf("Child Process. PID = %d\n", getpid());
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        printf("Parent Process. PID = %d\n", getpid());
        /* parent will wait for the child to complete */
        wait(NULL);
        printf("Child Complete\n");
    }
    return 0;
}
```

```

GNU nano 7.2      multiprocessing.c *
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

int main(){
    pid_t pid;

    /*fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /*child process */
        printf("Child Process. PID = %d\n", getpid());
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        printf("Parent Process. PID = %d\n", getpid());
        /* parent will wait for the child to complete */
        wait(NULL);
        printf("Child Complete\n");
    }
    return 0;
}

```

Output:

```

lam@secr2043:~$ nano multiprocessing.c
lam@secr2043:~$ gcc multiprocessing.c -o multiprocessing
lam@secr2043:~$ ./multiprocessing
Parent Process. PID = 875
Child Process. PID = 876
activity3      mainprocess      nano.959.save   subprocess2
bye.txt        mainprocess.c    os_lab          subprocess2.c
example.txt    multiprocessing  share
greeting.txt   multiprocessing  subprocess1
hello.txt      myfile.txt       subprocess1.c
Child Complete

```